

Sicurezza

CdL in Informatica – L31

Lezione 5 Access Control

Prof. Emiliano Casalicchio
Department of Computer Science
Sapienza University of Rome



SAPIENZA
UNIVERSITÀ DI ROMA



Argomenti trattati

- Access control paradigm (S.Graham and P.Denning)
- AC policies
 - Base concepts
- Implementation of AC
 - Access Control Directory, Access Control Matrix, ACL, Capabilities
 - Procedure oriented AC
 - Role Based AC
 - Attribute Based AC



Access Control (AC)

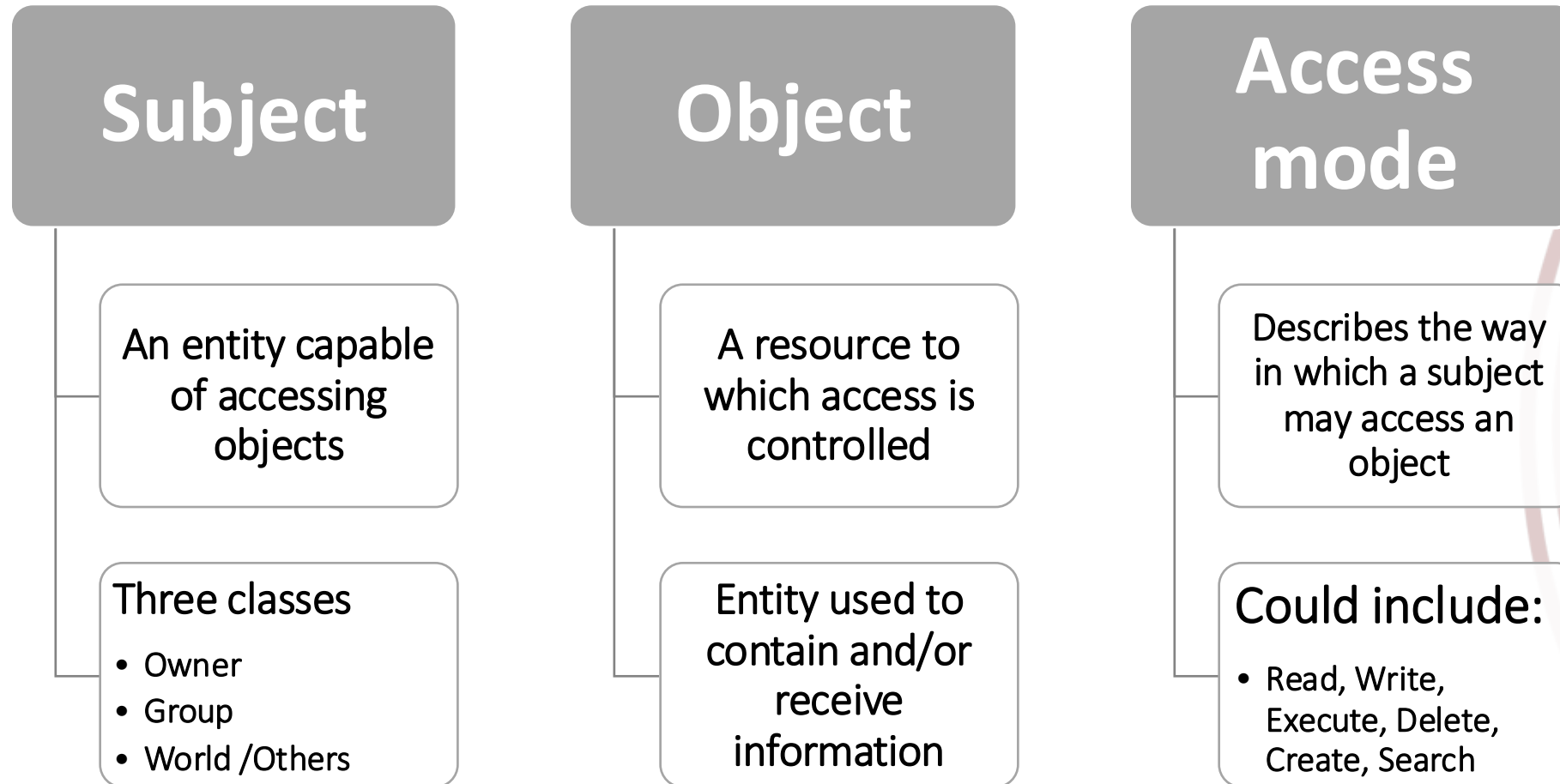
Is about **limiting who can access what in what ways**

The S.Graham and P.Denning (1972) AC paradigm states:

- A **subject** is permitted to access an **object** in a particular **mode (access mode)**, and only such authorized accesses are allowed
- **Objects** could be also a programs or processes representing users
- **Subject** could be programs running on behalf of the users
- **Access modes** are any controllable actions of subjects on objects

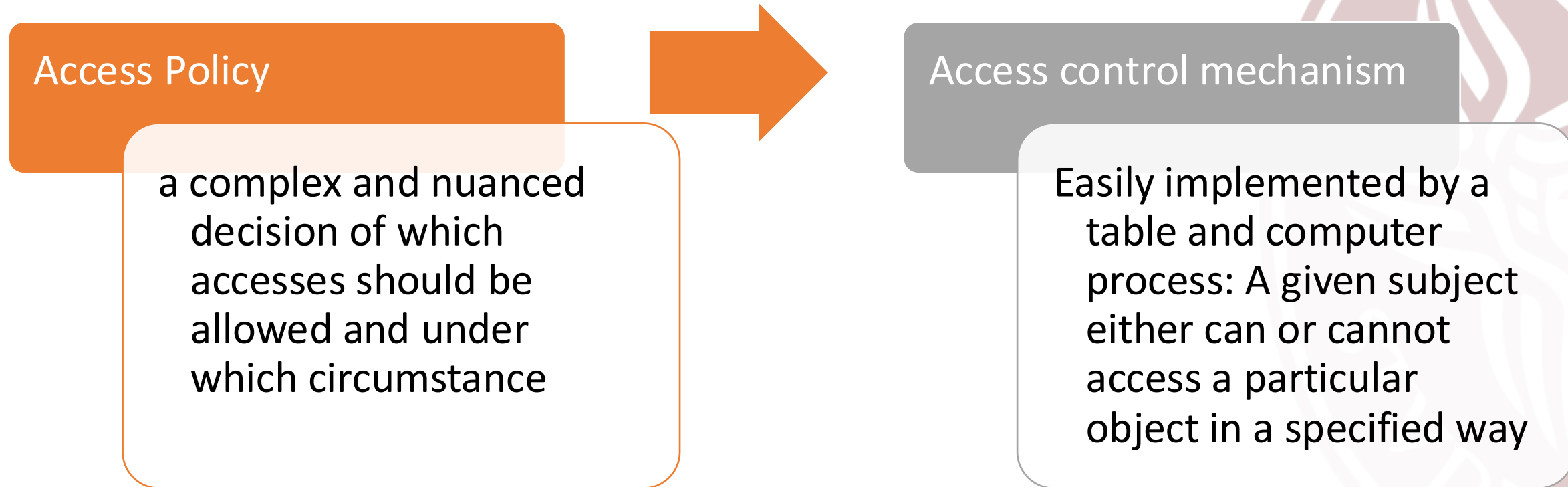


Basic elements of access control



Access policies (AP)

- Drive the implementation of access control mechanisms



AP: Goals of Protecting objects

Check every access right periodically

- to revoke a user's privilege to access an object
- To be sure the privilege revocation is enforced immediately

Enforce least privilege

- The principle of **least privilege** states that a subject should have access to the smallest number of objects necessary to perform some task
- Not allowing access to unnecessary objects guards against security weaknesses if a part of the protection mechanism should fail.

Verify acceptable usage

- Is important to check that the activity to be performed on an object is appropriate



AP: Tracking to verify protection goals

Need to revisit the access policy to determine whether it is working as it should, e.g.,

- Has someone been around for a long time and so has acquired a large number of no-longer-needed rights?
- Do so many users have access to one object that it no longer needs to be controlled? Or should it be split into several objects so that individuals can be allowed access to only the pieces they need?

Administering AP involves **tracking and reviewing access** in at least two ways:

- What objects and systems can a given user access?
- And for each object and system, who has access?



AP: Granularity

Fineness or specificity of access control

- No access → Complete access

The finer the granularity, the larger number of access control decisions that must be made, the higher is a performance penalty, e.g.

- A word or set of bytes in a document
- A cell or set of cells in a spreadsheet
- Each photo in a collection

Typically, a file, a program, or a data space is the smallest unit to which access is controlled.



AP: Access log

- Systems also record which accesses have been permitted, creating what is called an **audit log**
 - the log is preserved for later automated or manual analysis.
- Reasons for logging access include the following:
 - Records of accesses **can help plan for new or upgraded equipment** by showing which items have had heavy use
 - If the system fails, these records can show what accesses were in progress and perhaps **help identify the cause of failure**
 - If a user misuses objects, the access log shows exactly **which objects the user misuses**
 - In the event of an external compromise, the audit log may help identify **how the assailant gained access and which data items were accessed** (and therefore revealed or compromised).



AP: Limited privilege

- Is the act of restraining users and processes so that any harm they can do is contained in some way, so is not likely to be catastrophic
 - A system that prohibits all accesses to anything by anyone certainly achieves both confidentiality and integrity, but it completely fails availability and usefulness
- We seek a midpoint that balances the need for some access against the risk of harmful, inappropriate access
- Limited privilege **is a management concept**, not a technical control.
 - The process of analyzing users and determining the privileges they require is a necessary first step to authorizing within those limits.
 - After establishing the limits, we turn to access control technology to enforce those limits.



Implementing access control



Who does Access control

- Often performed by the operating system
 - Only the OS can access primitive objects, such as files, to exercise control over them, and the operating system creates and terminates the programs that represent users (subjects)
- However
 - current hardware design does not always support the operating system in implementing well-differentiated or fine-grained access control
 - The OS does not see the data object of applications e.g. DB
 - The OS cannot easily examine and discriminate network traffic
 - So, the AC is left to the application or network appliance



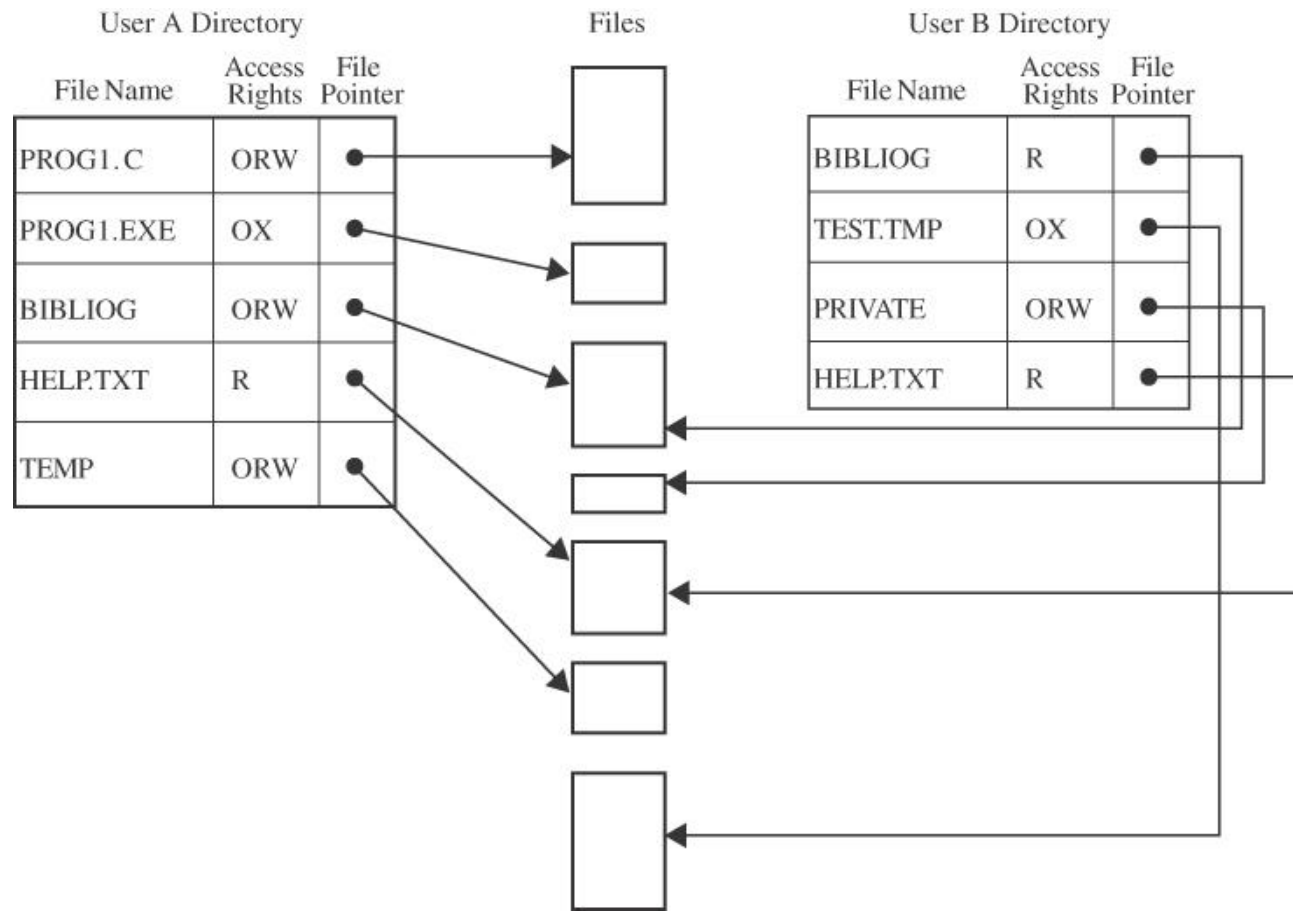
The Reference Monitor (RM)

J. Anderson, 1972

- Access control depends on a combination of hardware and software that is
 - **always invoked**; so that it validates every access attempt
 - **immune from tampering**
 - **assuredly correct**
- RM is a concept, not a plug and play tool !!
- To have an effective RM, we need to consider effective and efficient means to translate AC policies into action.



Access Control Directory



Assume we want to protect files

- Every file has a unique owner who possesses
 - The rights to declare who has what access
 - The right to revoke access of any person at any time.
- There is a Directory for each user
- No user can be allowed to write in the Access Control Directory because that would be a way to forge access to a file

Access Control Directory: pro & cons

PRO

- **Easy implementation:** a list for each user of all the object it can access

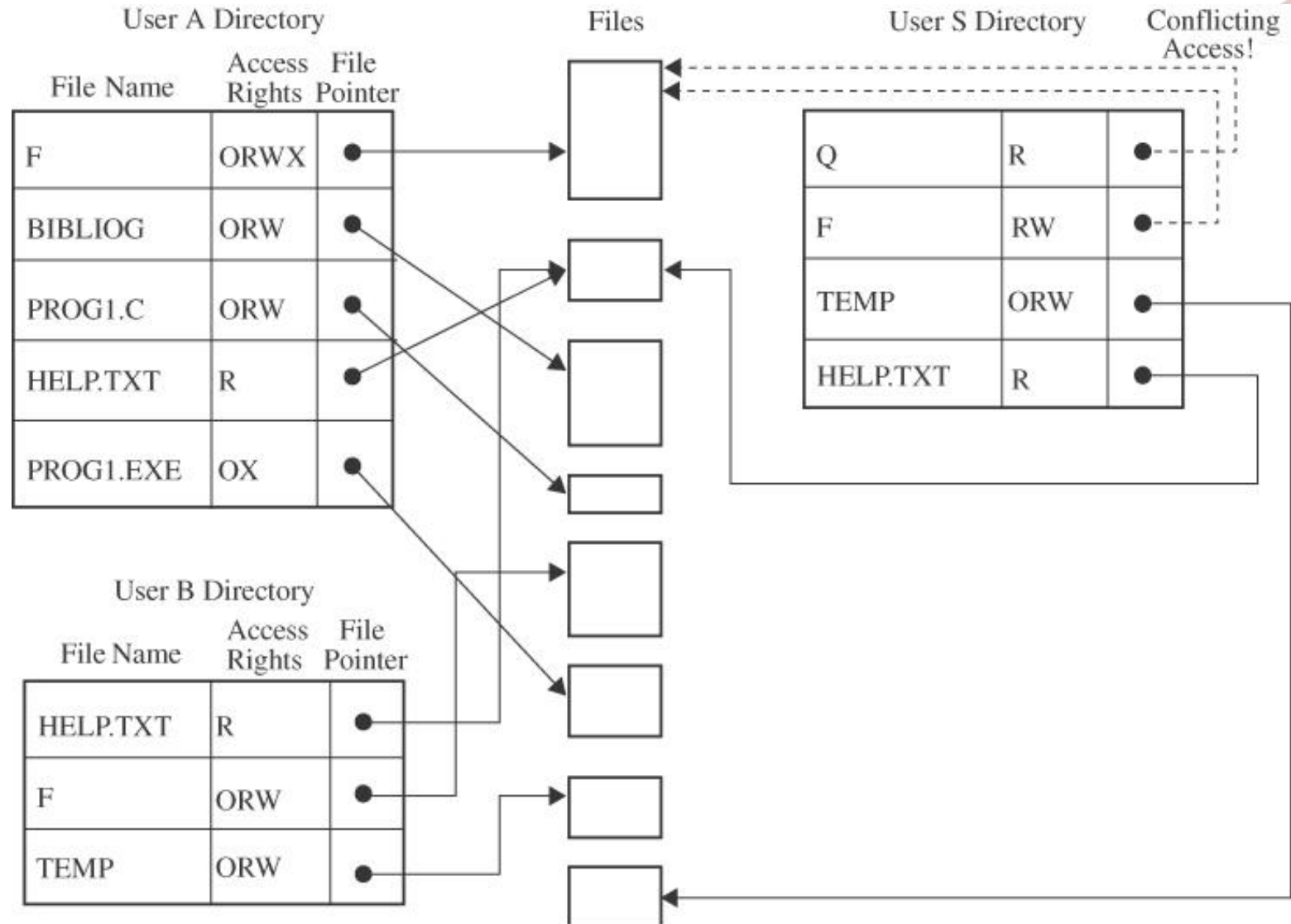
CONS

- **List size:** the list became too large, if many shared objects
- **Revocation of access:**
 - Cheap if A want to revoke permission to B, costly if A want to revoke permission to everyone
 - If A grant permission to B and B transfer permission to C, A may not know, this is the **propagation of access rights problem**
- **Pseudonyms** are a problem. If A own F also B can own a file called F, if both grant permission to C, the reference monitor must keep track of two different files with the same name accessible by C.
- **Renaming** of files is a problem.



Example of complicating access right after a file renaming

- S rename Q in F and then ask again A access to F A give more permission right.
- S has **inconsistent multiple permission** to access F



Access Control Matrix (ACM)

		objects		
subjects		File A	Printer	System Clock
	User W	Read Write Own	Write	Read
	Admin		Write Control	Control

	Bibliog	Temp	F	Help .txt	C_Com p	Linker	Clock	Printer
USER A	ORW	ORW	ORW	R	X	X	R	W
USER B	R	–	–	R	X	X	R	W
USER S	RW	–	R	R	X	X	R	W
USER T	–	–	–	R	X	X	R	W
SYS MGR	–	–	–	RW	OX	OX	ORW	O
USER SVCS	–	–	–	O	X	X	R	W

ACM: list of triples implementation

ACM is a sparse matrix:

- can be represented as a list of triples, each having the form $\langle \text{subject}, \text{object}, \text{rights} \rangle$
 - More efficient there is no triple for any empty cell of the matrix such as $\langle \text{USER T}, \text{Bibliog}, - \rangle$
- Triples can be sorted by **subject** or **object** as needed

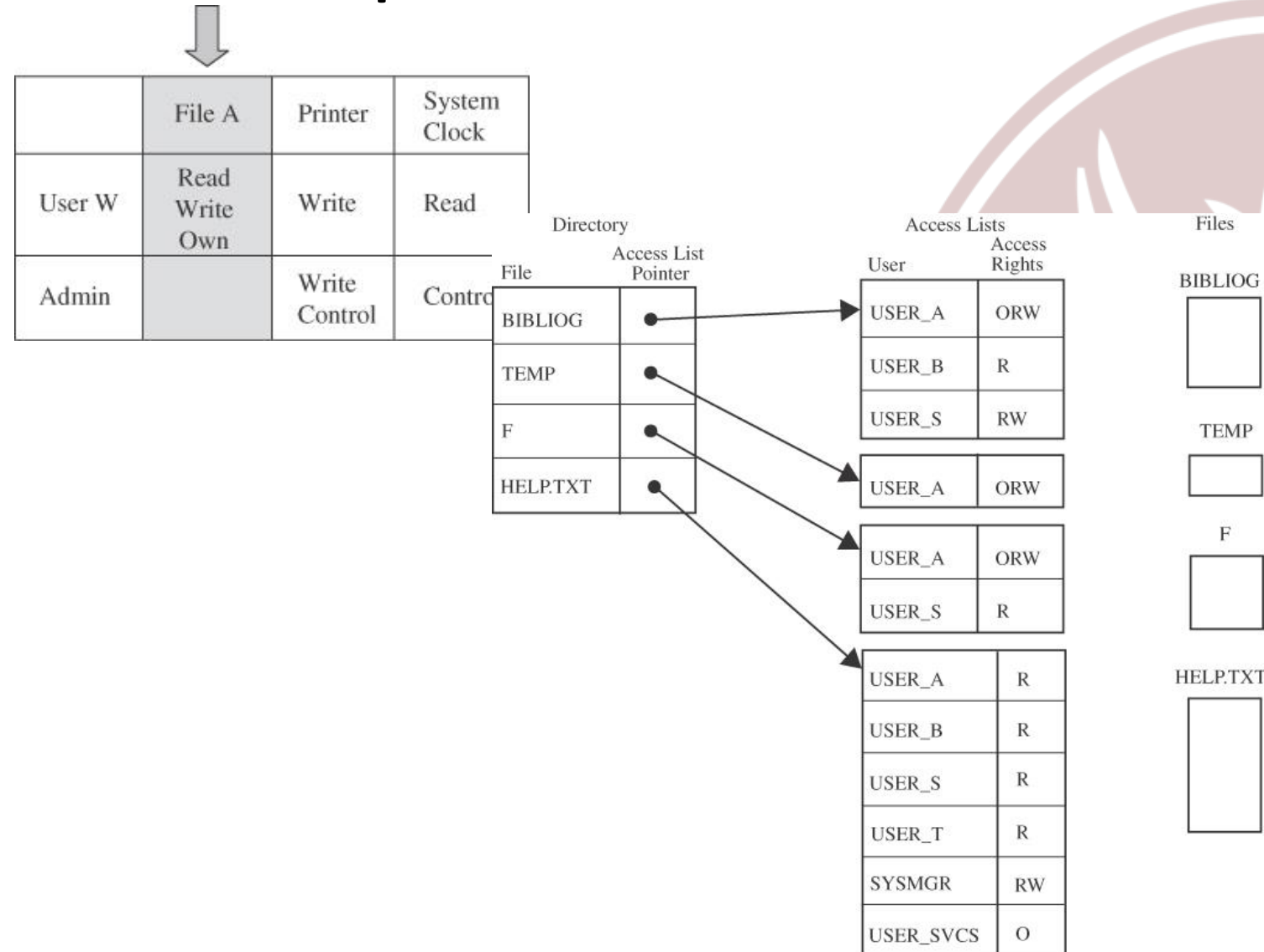
The problem: searching a large number of these triples is inefficient enough that this implementation is seldom used.



ACM: Access Control List implementation

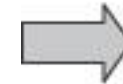
Columns of the ACM

- Can include default right
- Allow the use of wild cards (*)
 - Any user, any group, anyone else
- ACL with * sorted as coming after all specific names facilitate check
 - If no name match and there is a wild card, the access is granted, otherwise not
 - The last entry on an access list could be *-*-, that makes list of shared objects very short



ACM: Priviledge List implementation

- Rows of the ACM (also referred as **directory**)
- Advantage is ease of revocation:
 - If a user is removed from the system, the privilege list shows all objects to which the user has access so that those rights can be removed from the object



	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control

Consideration

Access Control Directory
and
Access Control Matrix



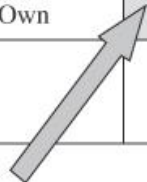
The Operating System (for files)
or
the **Reference Monitor** (in general)
*must keep track of all the
protection objects and rights*

Capability

Is an **unforgeable** token that gives the possessor certain rights to an object

Represented as an access control triple of a **subject**, **object**, and **right**.

	File A	Printer	System Clock
User W	Read Write Own	Write	Read
Admin		Write Control	Control



Making the token unforgeable

1) Giving the ticket directly to the OS (or AC mechanisms) and not to the user

- The OS holds all tickets on behalf of the users.
- The OS returns to the user a pointer to an OS data structure, which also links to the user
- A capability can be created only by a specific request from a user to the operating system
 - Each capability also identifies the allowable accesses.

2) By encryption with a key available only to the OS or AC mechanism

- If the encrypted capability contains the identity of its rightful owner, user A cannot copy the capability and give it to user B



Propagation of access right with capabilities

- With capabilities we introduce the transfer or **propagate** right
- A subject having propagate right can pass copies of capabilities to other subjects.
 - Each of these capabilities also has a list of permitted types of accesses, one of which might also be transfer
- Example: process A can pass a copy of a capability to B, who can then pass a copy to C. B can prevent further distribution of the capability (and therefore prevent further dissemination of the access right) by omitting the transfer right from the rights passed in the capability to C. B might still pass certain access rights to C, but not the right to propagate access rights to other subjects



Revocation of capabilities

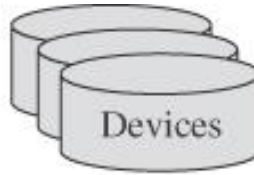
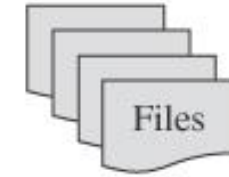
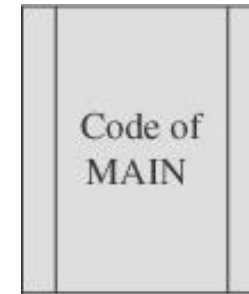
When an issuing subject **revokes** a capability, no further access under the revoked capability should be permitted.

- A capability table can contain pointers to the active capabilities spawned under it so that the operating system can trace what access rights should be deleted if a capability is revoked.

A similar problem is deleting capabilities for users who are no longer active.

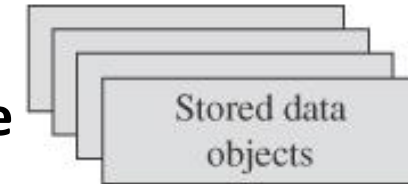


Domain



Is the collection of objects to which the process has access

- The **collection of capabilities** of the objects in the domain **define the domain**



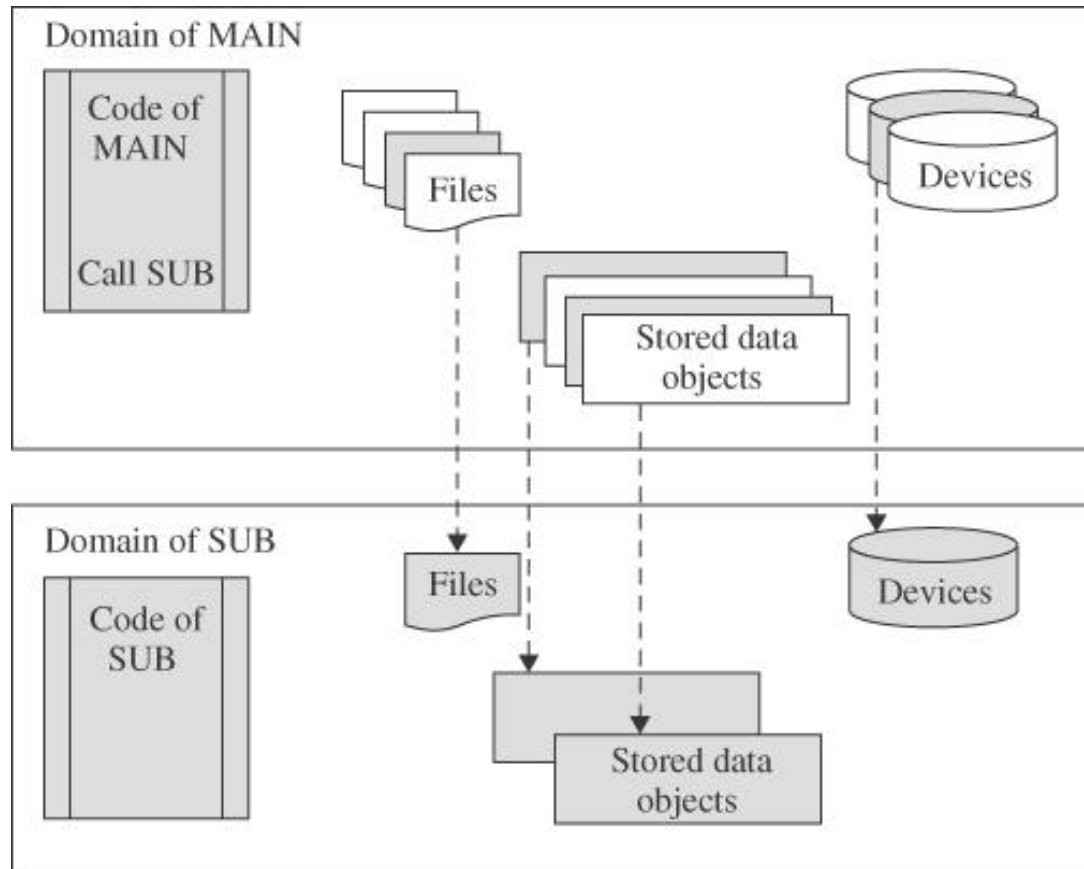
The process run on behalf of a user

- A **domain for a user** at a given time might include some programs, files, data segments, and I/O devices such as a printer

The *namespace* of a process in Linux is an example of domain



Passing objects through Domains



The domain of the subprocedure is not necessarily the same as that of its calling procedure

- the calling procedure may pass only some of its objects to the subprocedure,
- the subprocedure may have access rights to other objects not accessible to the calling procedure
- the calling procedure may pass only some of its access rights for the objects it passes to the subprocedure

Capabilities at work

Capabilities are a straightforward way to keep track of the access rights of subjects to objects during execution

- The capabilities require the use of a ACL or ACM
- A capability for an object is created at runtime
 - Each time a process seeks to use a new object, the OS examines the master list of objects and subjects to determine whether the object is accessible. If so, the OS creates a capability for that object.



Where to store capabilities

Must be stored in memory inaccessible to normal users:

- in segments not pointed at by the user's segment table
- in protected memory as from a pair of base/bounds registers
- using a tagged architecture machine to identify capabilities as structures requiring protection

During execution, only the capabilities of objects that have been accessed by the current process are kept readily available



Procedure-oriented Access Control



Procedure-oriented protection

Imply the existence **of a procedure that controls access to objects**

- for example, by performing its own user authentication to strengthen the basic protection provided by the basic operating system

The procedure forms a capsule around the object, permitting only certain specified accesses

- Procedures can ensure that accesses to an object be made through a trusted interface
 - E.g. The access to the table of valid users could be only through three procedures: add a user, delete a user, check whether a particular name corresponds to a valid user

It **implements the principle of information hiding** because the means of implementing an object are known only to the object's control procedure



Role-Based Access Control



RBAC is based on the roles that users assume in a system rather than the user's identity.

RBAC models define a role as a job function within an organization.

RBAC systems assign access rights to roles instead of individual users.

Users are assigned to different roles, either statically or dynamically, according to their responsibilities.

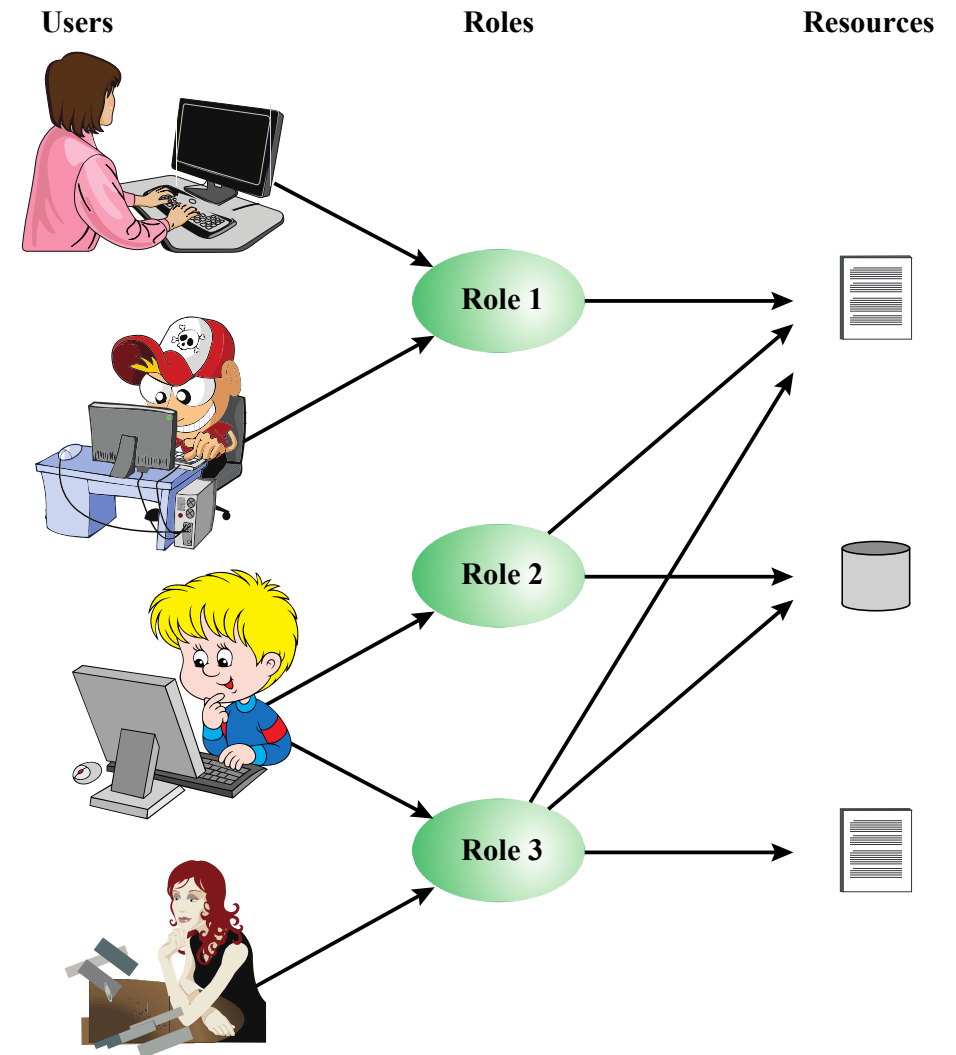
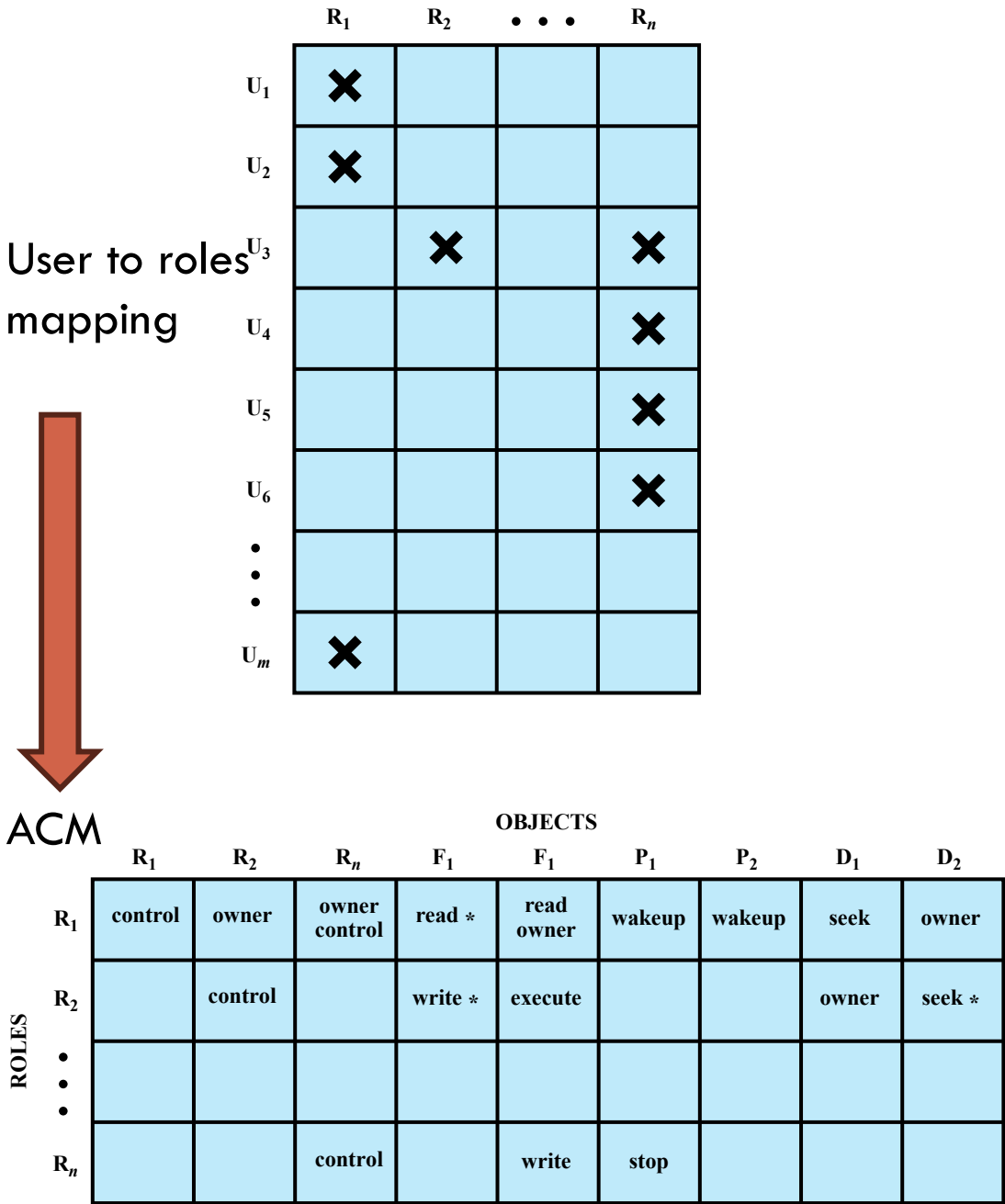


Figure 4.6 Users, Roles, and Resources

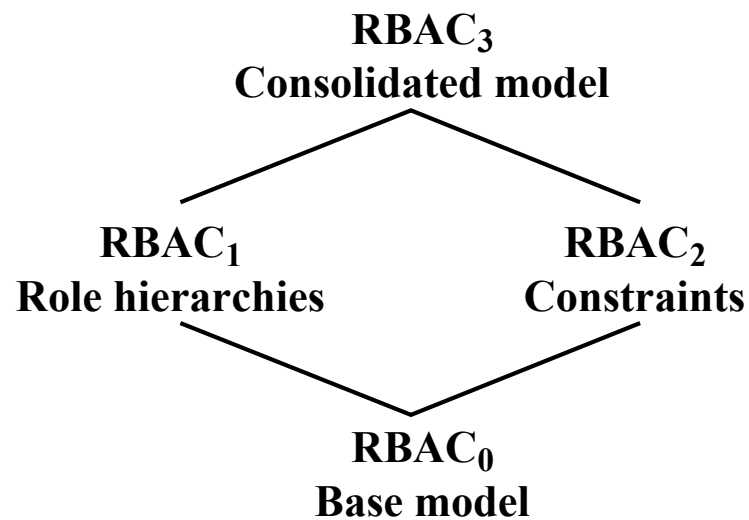
RBAC lends itself to an effective implementation of the principle of least privilege

Each role should contain the minimum set of access rights needed for that role.

A user is assigned to a role that enables him or her to perform only what is required for that role.



Modelli RBAC

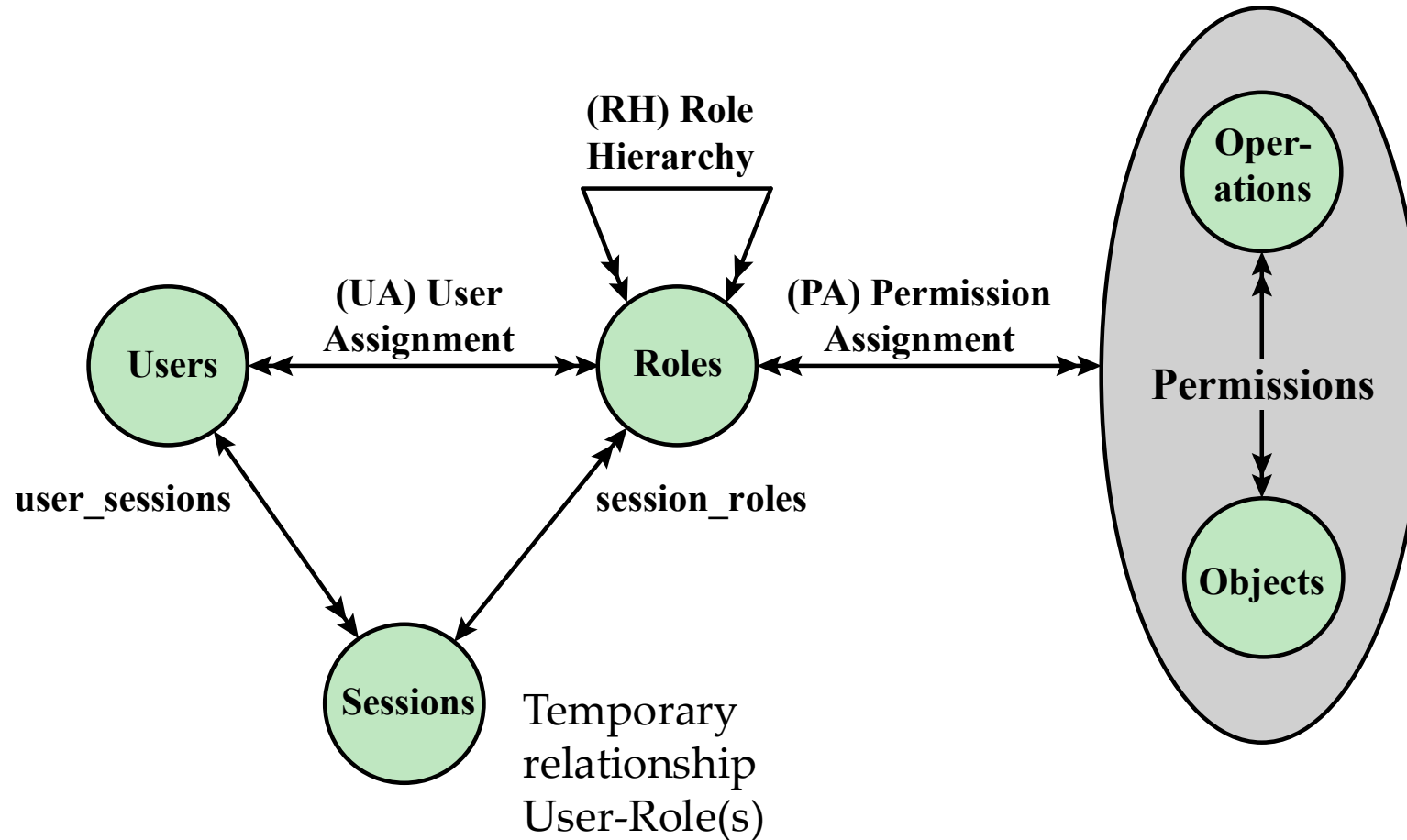


(a) Relationship among RBAC models

Role hierarchies enable one role to inherit permissions from another role

Constraints restrict the ways in which the components of a RBAC system may be configured

RBAC models



(b) RBAC models

RBAC Hierarchy

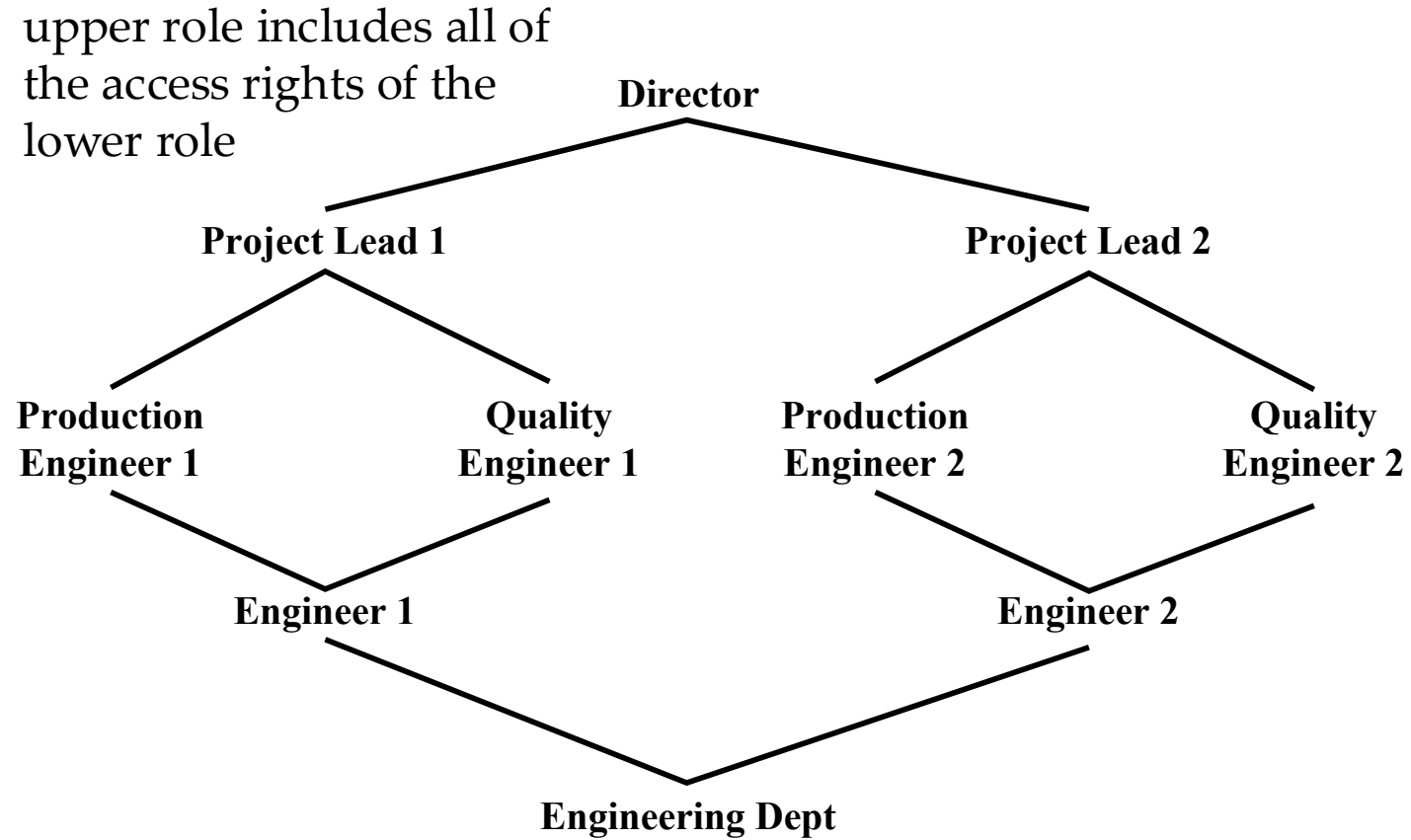


Figure 4.9 Example of Role Hierarchy

Constraints - RBAC

- Provide a means of adapting RBAC to the specifics of administrative and security policies of an organization
- A defined relationship among roles or a condition related to roles
- Types:

Mutually exclusive roles

- A user can only be assigned to one role in the set (either during a session or statically)
- Any permission (access right) can be granted to only one role in the set

Cardinality

- Setting a maximum number with respect to roles
 - Max #User to role
 - Max #roles to permission

Prerequisite roles

- Dictates that a user can only be assigned to a particular role if it is already assigned to some other specified role

Attribute-Based Access Control (ABAC)



ABAC

Can define authorizations that express conditions on properties of both the resource and the subject

Strength is its flexibility and expressive power

Concern about the performance impact of evaluating predicates on both resource and user properties for each access

Web services have been pioneering technologies through the introduction of the eXtensible Access Control Markup Language (XAMCL)

There is considerable interest in applying the model to cloud services

ABAC Model: Attributes

Subject attributes

- A subject is an active entity that causes information to flow among objects or changes the system state
- Attributes define the identity and characteristics of the subject
- e.g. Role, Department, Job Title, Clearance Level

Object attributes

- An object (or resource) is a passive information system-related entity containing or receiving information
- Objects have attributes that can be leveraged to make access control decisions
- e.g. File Type, Data Sensitivity, Owner, Project Association

Environment attributes

- Describe the operational, technical, and even situational environment or context in which the information access occurs
- These attributes have so far been largely ignored in most access control policies
- e.g. Time of Day, Network Security Level, Location

ABAC

Distinguishable because it controls access to objects by evaluating rules against the attributes of entities, operations, and the environment relevant to a request

Relies upon the evaluation of attributes of the subject, attributes of the object, and a formal relationship or access control rule defining the allowable operations for subject-object attribute combinations in a given environment

Allows an unlimited number of attributes to be combined to satisfy any access control rule

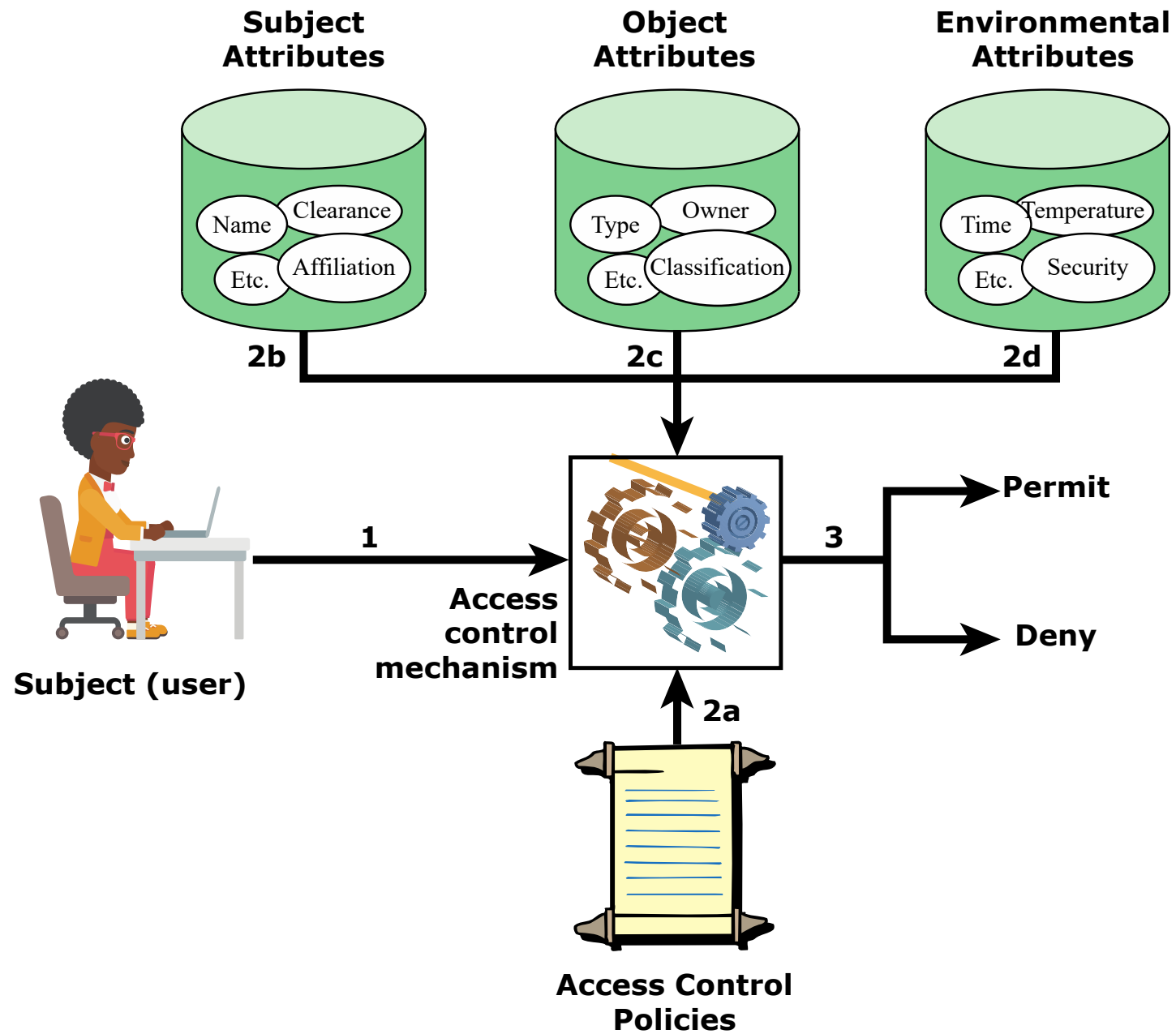


Figure 4.10 ABAC Scenario

ABAC examples

Time-Based Access:

- A "Sales" employee (subject) can only edit "Client CRM records" (object) if the "Current Time" (environment) is between 08:00 and 18:00.

Location-Based Access:

- An employee can access "Sensitive Data" only if their "Connection Location" is "Internal Corporate Network" (environment).

Dynamic Role/Project Access:

- An engineer (subject) can view "Project X Files" (object) only if the engineer's "Project Assignment" attribute matches the file's "Project Tag".

Financial Transaction Approval:

- A financial analyst can approve a transfer, but only if the "Transaction Value" (object) is less than their "Authorization Limit" (subject).

Contractor Access:

- A consultant can access a document, but their "Employment Status" is "Contractor" and their access is only permitted until the "Expiration Date" (environment).

Region-Specific Data Visibility:

- A regional employee can view rows in a database only if the "Region Tag" on the data matches their own "Region Attribute," such as 'region=EMEA'.

Summary

- Access control paradigm (books CS)
- AC policies (books CS)
 - Base concepts
- Implementation of AC (books CS + CSPP)
 - Reference Monitor
 - Access Control Directory, Access Control Matrix, ACL, Capabilities
 - Procedure oriented AC
 - Role Based AC. (books CSPP + SC)
 - Attribute Based AC (book CSPP)

